

Tarea 5

Índice

1. Problema 1	1
1.1. Solución	1
1.1.1. Ejemplo	2
2. Problema 2	2
2.1. Solución	3
2.1.1. Ejemplo	3

1. Problema 1

Tenemos un banco de n problemas algorítmica p_i para $i = 1, \dots, n$.

Después de un trabajo de análisis profundo de estos problemas, has logrado establecer una lista de m reducciones de problema q en problema p :

$$p \rightarrow q$$

significando que, si tienes un algoritmo que permite resolver p , entonces sabrás resolver q con el mismo algoritmo.

Escribir un programa que determine el número mínimo $n_{\min}$ de algoritmos que escribir para que todos los problemas del banco de problemas se puedan resolver, o directamente, o por la la reducción. Por ejemplo, si tienes tres problemas 1, 2, 3 y que sabes que 3 se reduce a 1, entonces bastará tener 2 algoritmos para resolver los 3, y eso es el mínimo.

El programa deberá leer m, n así como una lista de reducciones $p_i \rightarrow q_i$ donde $1 \leq p_i, q_i \leq n$. Deberá poder manejar hasta 10^5 problemas (identificados por enteros positivos) y 10^5 especificaciones de reducciones.

Entrada

$n \ m$

$p_1 \ q_1$

$p_2 \ q_2$

$p_3 \ q_3$

$\vdots \ \vdots$

$p_m \ q_m$

Salida

$n_{\min}$

1.1. Solución

Tenemos n nodos que representan problemas distintos. Construimos un grafo dirigido donde las aristas representan las reducciones dadas. Al obtener este grafo, generamos el grafo condensado que resulta a partir del grafo cociente sobre la relación de equivalencia dada por las componentes fuertemente conexas. Es decir, el resultado es un DAG. Cada nodo del DAG representa 1 algoritmo distinto. Las aristas representan una reducción, que no necesariamente genera una equivalencia. Entonces, buscamos los nodos que no tienen ancestro.

1.1.1. Ejemplo

Supongamos que tenemos la entrada

```
8 12
1 5
1 6
5 6
3 1
3 8
6 3
6 8
8 2
2 8
4 8
8 7
7 2
```

Nos genera el siguiente grafo dirigido:

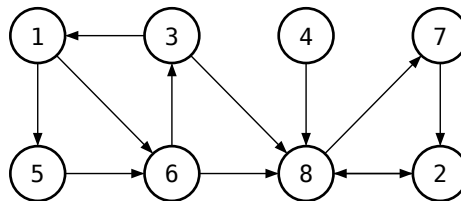


Figura 2: Grafo dirigido inicial.

El grafo condensado resultante es el siguiente:

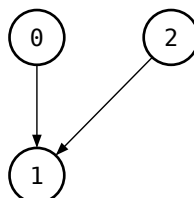


Figura 3: Grafo condensado.

Dado que solo hay 2 nodos sin ancestros, la respuesta es 2.

Para ejecutar y verificar nuestro programa con esta entrada, compilamos con

```
g++ p1.cpp -o p1
```

Y ejecutamos con

```
./p1
```

Luego, insertamos la entrada previamente expuesta.

2. Problema 2

Re-escribe el algoritmo de ordenamiento topológico para poder recuperar la lista de todos los ordenamientos topológicos posibles. El algoritmo leerá la descripción de un DAG y dará la lista de los ordenamientos topológicos posibles.

Entrada

$n \ m$

$i_1 \ e_1$

$i_2 \ e_2$

$i_3 \ e_3$

$\vdots \ \vdots$

$i_m \ e_m$

Salida

Impresión de **todos** los ordenamientos topológicos posibles

2.1. Solución

Para encontrar todos los ordenamientos topológicos, usaremos el algoritmo de Kahn, que se basa en usar BFS. Buscamos los nodos sin ancestros, los agregamos a una lista, y los borramos del grafo. Para obtener todos los ordenamientos, utilizamos backtracking.

2.1.1. Ejemplo

Supongamos que tenemos el siguiente grafo dirigido:

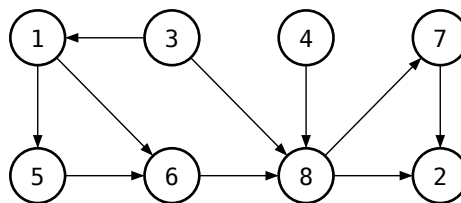


Figura 5: Grafo dirigido.

Es decir, tenemos el input

```

8 10
1 5
1 6
5 6
3 1
3 8
6 8
8 2
4 8
8 7
7 2
  
```

Esto nos genera los siguientes ordenamientos:

```

3 1 4 5 6 8 7 2
3 1 5 4 6 8 7 2
3 1 5 6 4 8 7 2
3 4 1 5 6 8 7 2
4 3 1 5 6 8 7 2
  
```